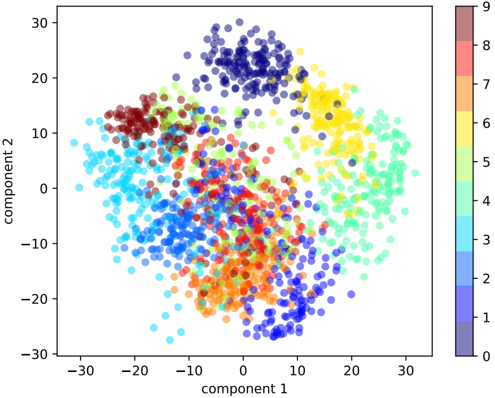
Machine learning Tek 5: dimensionality reduction



Overview

Motivation

Linear dimensionality reduction (Principal component analysis)

Nonlinear dimensionality reduction (manifold learning)

Motivation

Linear dimensionality reduction (Principal component analysis)

Nonlinear dimensionality reduction (manifold learning)

Why is the dimension so important ?

Recall 2 important constants in machine leaning :

- n : number of samples
- d : dimension of the samples (number of features)

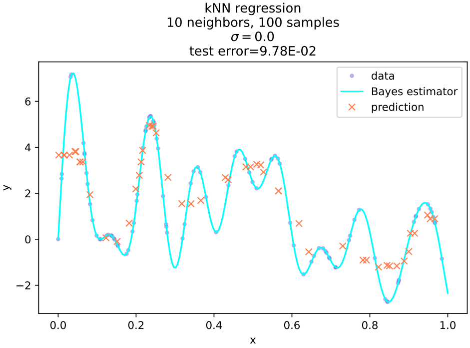
The space $\mathcal{X}$ that contains the data, for either supervised or unsupervised learning is often included in $\mathbb{R}^d$.

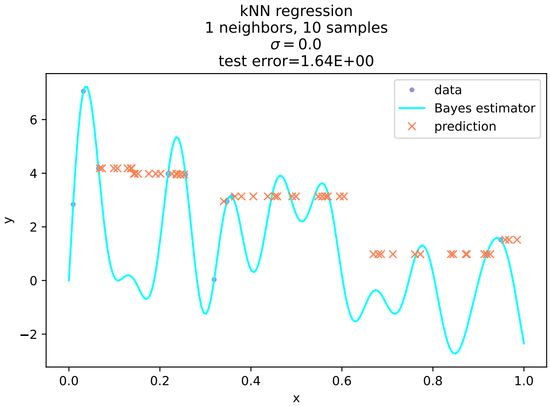
Dimensionality reduction

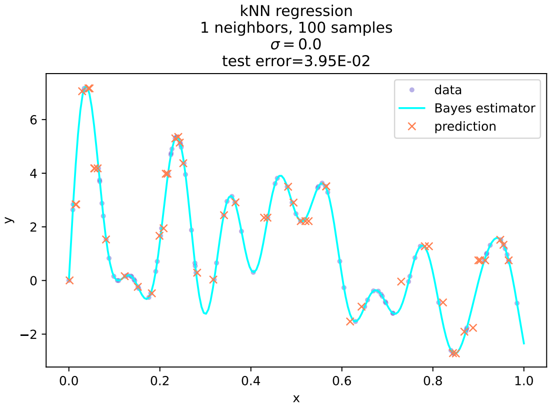
- $\mathcal{X} \subset \mathbb{R}^d$.
- If d is large (e.g. $\geq 10^4$), the algorithms that run on the data might become too slow to be used, as their algorithmic complexity depends on d (potentially in a quadratic or exponential way, curse of dimensionality)
- we will illustrate this problem with local averaging.

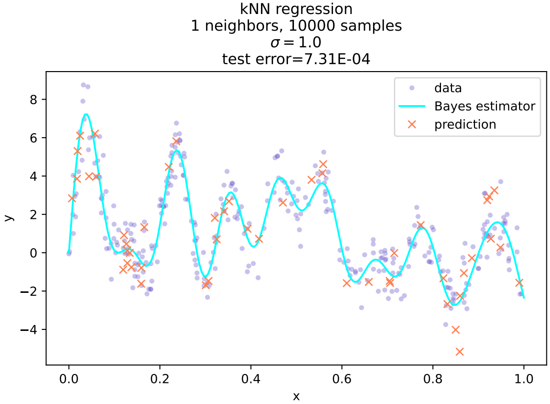
Local averaging

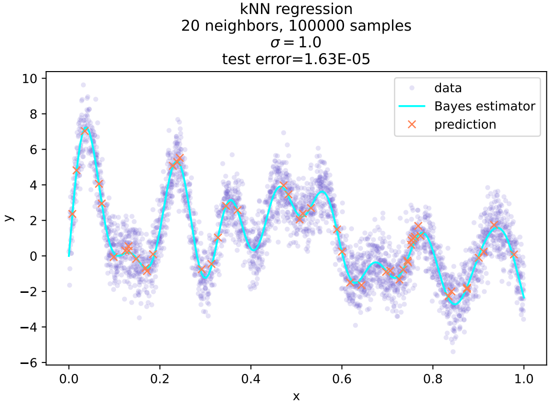
Local averaging is a prediction method that is not based on any opitmization, but only based on the dataset itself.




kNN regression
1 neighbors, 10 samples
σ = 0.0
test error=1.64E+00
data
Bayes estimator
prediction
y
x
6
4
2
0
−2
0.0
0.2
0.4
0.6
0.8
1.0


kNN regression
1 neighbors, 100 samples
σ = 0.0
test error=3.95E-02
data
Bayes estimator
prediction
y
x
6
4
2
0
−2
0.0
0.2
0.4
0.6
0.8
1.0


kNN regression
1 neighbors, 10000 samples
σ = 1.0
test error=7.31E-04
data
Bayes estimator
prediction
8
6
4
2
0
−2
−4
y
0.0
0.2
0.4
0.6
0.8
1.0
x


kNN regression
20 neighbors, 100000 samples
$\sigma = 1.0$
test error=1.63E-05
data
Bayes estimator
prediction
y
x
10
8
6
4
2
0
−2
−4
−6
0.0
0.2
0.4
0.6
0.8
1.0

The problem with local averaging is that the number of samples that we need in order to garantee a prediction error of order of magnitude ϵ is

$$n_\epsilon \geq \frac{\epsilon^{-d} d^{d/2}}{\alpha^{d/2}} \tag{1}$$

where $\alpha >$ is a constant. n_ϵ hence grows exponentially with $1/\epsilon$, and this is an example of curse of dimensionality.

Dimensionality reduction

- Hence, in general, it is not possible to use local averaging when the input space dimension is larger than e.g. 20.
- **However**, often the data might actually occupy a **subspace** of lower dimension q, or it may be possible to project the data on such a subspace without loosing too much information.
  - Working in a subspace of lower dimension might speed up the algorithms.
  - It may also allow visualization of the data.

Methods of dimensionality reduction

- **feature selection** : selecting a subset of the original dimensions.
- **feature extraction** : computing new features from the original features.

Motivation

Linear dimensionality reduction (Principal component analysis)

Nonlinear dimensionality reduction (manifold learning)

https://scikit-learn.org/stable/modules/unsupervised_reduction.html
https://en.wikipedia.org/wiki/Principal_component_analysis

Introduction

- Principal Component Analysis is a linear dimensionality reduction method.
- It is based on statistical and geometrical considerations.
- It was invented by Pearson at the beginning of XXth century but is still used today.
- Applications include :
  - dimensionality reduction
  - noise filtering
  - prediction
  - general data visualization and analysis

Example

In this paper, astrophysicists use PCA in order to test a new star temperature prediction method [Bermejo et al., 2013]

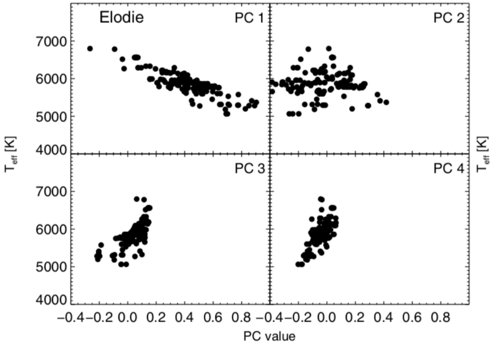
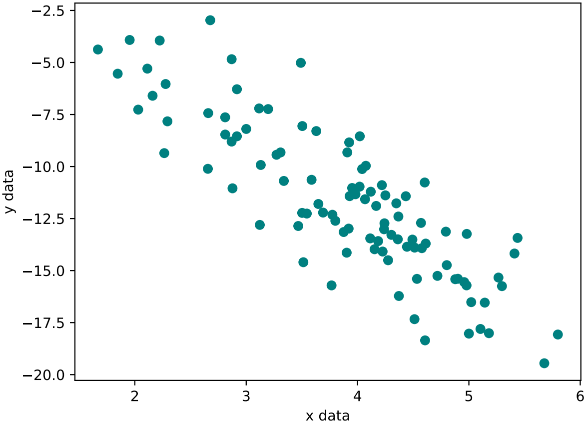


Figure – PCA used in order to predict temperature.

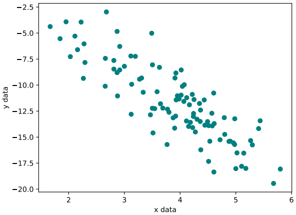
Problem statement

- We have multidimensional data.



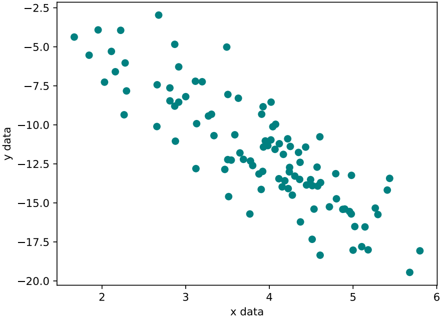
Problem statement

- We have multidimensional data.



We look for the axes that "explain" or carry the most variations in the data.
Thoses axes will be called the **principal components.**

Visual intuition



When looking at this image, what axis would you suggest in order to carry the biggest variation among the data ?

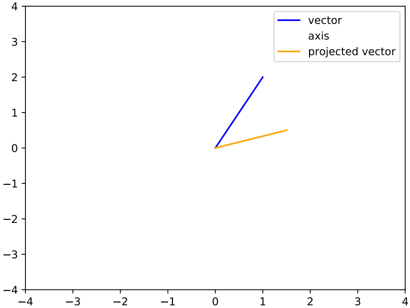
Formalization

We need a **mathematical criterion** in order to formalize this intuition.
In the case of a larger problem with more dimensions, it will not be possible to use visual feedback in order to choose the principal components (the axis).
Furthermore, even in 2D, using only visualization, we can only do a **approximation** of the most relevant axis.

Orthogonal projections

We will restate the problem in a mathematical way, using tools coming from **linear algebra.**
In particular, the concept of **orthogonal projection** is essential.

Orthogonal projection



Exercice 1 : **Mathematical problem** Can you link orthogonal projections to the problem of finding the most relevant axis ?

Inertia

- The **inertia** related to an axis will be a measure of the quality of an axis.
- If x_i is a sample from n samples, and Δ an axis, we call $p_{x_i \to \Delta}$ the orthogonal projection of x_i on the axis Δ.
- The inertia related to Δ is :

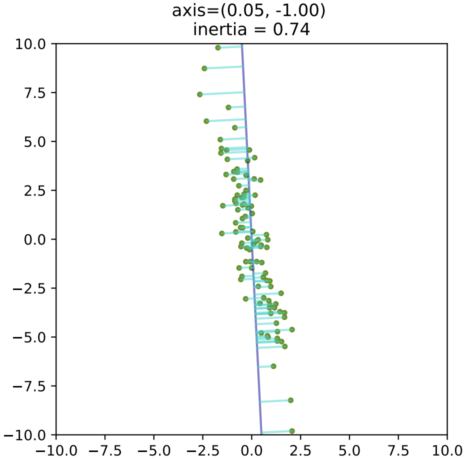
$$I_\Delta = \frac{1}{n} \sum_{i=1}^{n} d^2(x_i, p_{x_i \to \Delta}) \tag{2}$$

Inertia

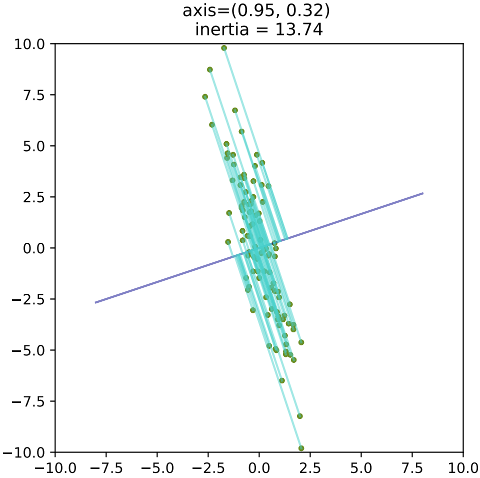
Exercice 2 : **No inertia**

In what situations could we have $I_\Delta = 0$?

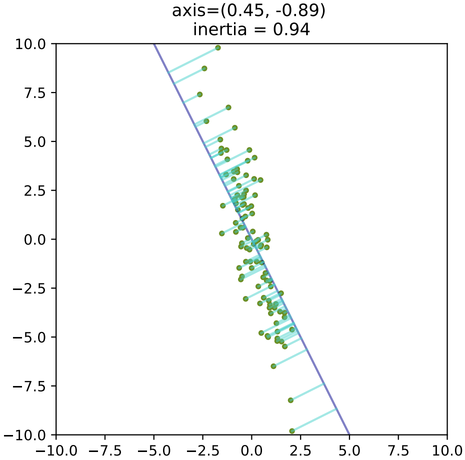
Inertia


axis=(0.05, -1.00)
inertia = 0.74
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−10.0 −7.5 −5.0 −2.5 0.0 2.5 5.0 7.5 10.0

Inertia


axis=(0.95, 0.32)
inertia = 13.74
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0

Inertia


axis=(0.45, -0.89)
inertia = 0.94
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0

Inertia

Exercice 3 : **Computing an inertia**
cd day_4/1_pca/toy_data/ and use the file **inertia.py** in order to represent the projections of the data onto a chosen axis and to compute the inertia of this axis.
You need to add the computation of the inertia to the script (starting at line 75).
What is your optimal axis ?

Remark

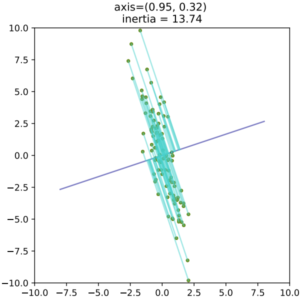
Minimizing I_Δ is the same as maximizing I_{Δ^*} where Δ^* is the supplementary orthogonal space.

Several principal components

- In $2D$ (like in the former example), we have at most 2 principal component.
- However, when the data have more dimensions, it is possible to have **several principal components** (as many as the dimensionality of the data).
- the data are then projected on these components.

Link with expected values

There is a connection between the inertia and the **statistical variance** of a specific random variable.



Expected value (espérance)

- For a discrete random variable X that takes the values x_i with probability p_i :

$$E(X) = \sum_{i=1}^{n} p_i x_i \tag{3}$$

- For a continuous random variable X with density p(x) :

$$E(X) = \int x p(x) dx \tag{4}$$

- Variance :

$$var(X) = E\left(\left(X - E(X)\right)^2\right) \tag{5}$$

Probability vs Statistics

(Advanced notions)

- from a dataset of samples $(x_1, \ldots, x_n)$, we can only compute **statistics**
- for instance
  - sample mean :

$$\hat{x} = \frac{1}{n} \sum_{i=1}^{n} x_i \tag{6}$$

  - unbiased sample variance :

$$\frac{1}{n-1} \sum_{i=1}^{n} (x_i - \hat{x})^2 \tag{7}$$

The $\frac{1}{n-1}$ factor instead of $\frac{1}{n}$ requires some theory to be properly understood.

Optimization with analytic solution

- In practical applications, algorithms or analytic solutions are used in order to find the optimal axes.
- Methods include the **Lagrange multiplier method**
- The method shows that we need to compute the **eigenvectors** of the **covariance matrix.**
- See also : Gram matrix, power iteration algorithm

PCA with scikit-learn

- https://scikit-learn.org/stable/modules/decomposition.html
- https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html

We can use **pca_sklearn.py** in order to find the principal components on the dataset of the previous exercise, using scikit-learn.

Iris dataset

We can also perform PCA on the iris dataset as in the file **1_pca/iris/iris_pca.ipynb**.

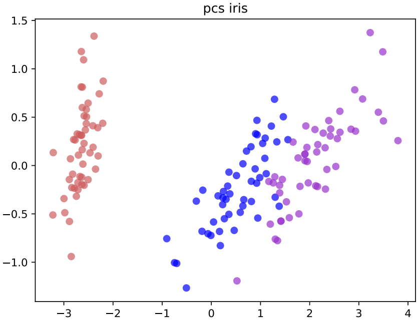
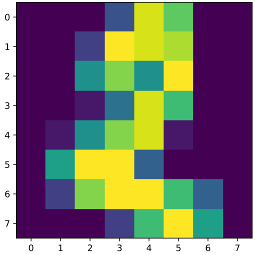


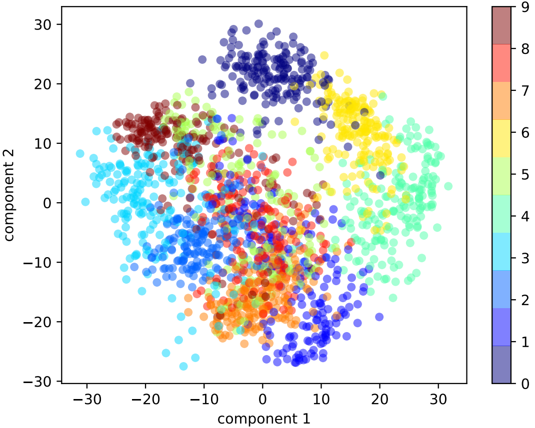
Figure – PCA performed on the iris dataset. We see that the principal components are (almost) able to separate the 3 classes.

PCA on digits

- We can apply PCA to the digits dataset, in **1_pca/digits/**, using **pca_digits.py**



PCA on digits

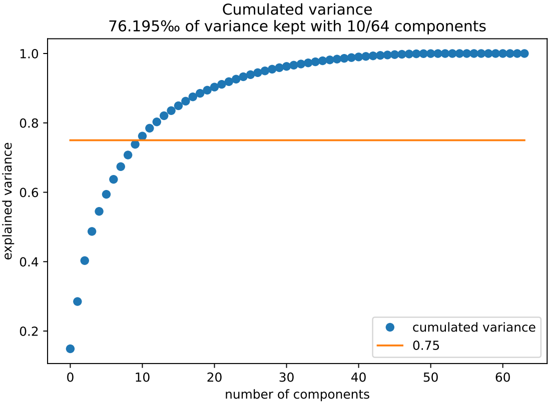


Number of components

Exercice 4 : **Choosing the number of components**

- The number of kept components for PCA is the most important hyperparameter.
- Use the file **pca_digits_variance.py** in order to determine how many components are necessary in order to keep 75% of the variance in the digits dataset.

Number of components



Reconstruction

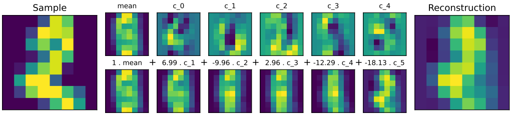


Figure – Reconstruction of a sample with 5 components

Reconstruction

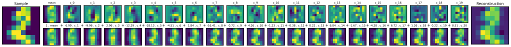
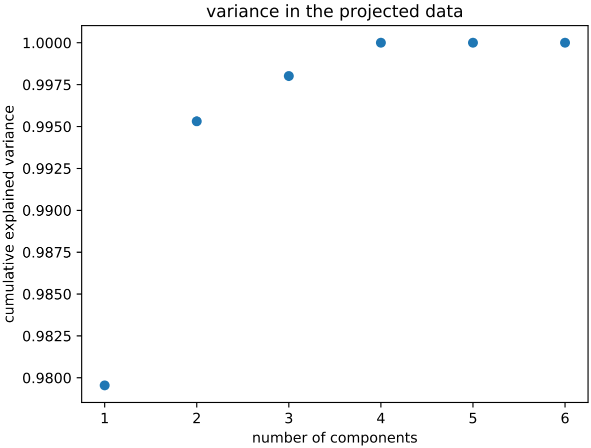


Figure – Reconstruction of a sample with 20 components

Number of components

Exercice 5 : **Redundancy**
What happens with the data contained in **redundancy/redundant_data.npy** ? You can analyze them with the file **redundancy/pca_variance.py**.

Number of components



Number of components

Conclusion : PCA can help detemine whether some components carry no information in the data.

Shortcomings of PCA

PCA is sensitive to :

- outliers
- initial data scaling
- Many variants and heuristics exist on this topic.

Asset of PCA

Reducing the dimensionality of the data might be very helpful in a situation where you need to train a supervised algorithm on the projected data. The algorithm might be way faster on data that live in a smaller space. However, a sufficient amount of information should be kept during the reduction, hence the necessity of experimentation and knowledge of heuristics.

https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_digits.html

Motivation

Linear dimensionality reduction (Principal component analysis)

Nonlinear dimensionality reduction (manifold learning)

https://scikit-learn.org/stable/modules/manifold.html
https://en.wikipedia.org/wiki/Nonlinear_dimensionality_reduction

Manifolds

Fundamental hypothesis (as before) :
In some cases, altough the dimensionality of the data is high (as for images), the data lie on a specific subpart of the linear space containing them.
This "subpart" is assumed to be a **manifold** (variété).

Manifolds

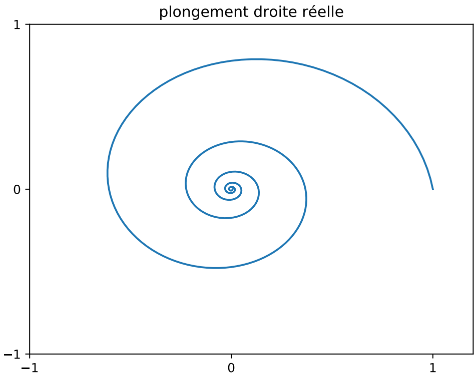


Figure – 1 dimensional manifold, in $\mathbb{R}^2$

Linearity

- In the case of PCA, the manifolds are **linear subspaces** of the ambient space : PCA is a **linear method** and does **linear projections.**
- A linear mapping in a linear space E is a mapping u such that

$$\forall x, y \in E, u(x + y) = u(x) + u(y) \tag{8}$$

Nonlinearity

- However, in many situations, the data might lie on **nonlinear** manifolds.
- **Manifold learning** is the unsupervised learning of these manifolds in order to study the structure of the data.

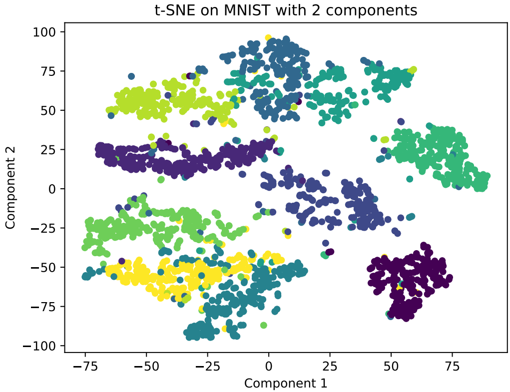
Multidimensional scaling (MDS) / Positionnement multidimensionnel

- Projects the data in a smaller subspace trying to preserve pairwise similarities between points.

t-SNE

- t-Stochastic Neighbor Embedding [Van Der Maaten and Hinton, 2008]
- `https://lvdmaaten.github.io/tsne/`

t-SNE on MNIST



Seel also

- Isomap
- LLE

Comments

Some disadvantages of non linear manifold methods :

- Hard to determine a good output dimension (whereas in PCA we can use explained variance) and it is hard to interpret the embedded dimensions (whereas in PCA we know what they mean).
- Depends on the number of neighbors chosen (if relevant)
- Often compuationally slower.

References

Bermejo, J. M., Ramos, A. A., and Prieto, C. A. (2013).
Astrophysics A PCA approach to stellar effective temperatures.
95 :1–9.

Van Der Maaten, L. J. P. and Hinton, G. E. (2008).
Visualizing high-dimensional data using t-sne.
Journal of Machine Learning Research, 9 :2579–2605.